RAČUNALNIK V POUKU FIZIKE

Zbirka laboratorijskih vaj za študente UL,PeF

dr. David Rihtaršič

2022-February

Kazalo

1 MERILNI SISTEMI **1**
1.1 Importance of real-life experiments for students . . . 1
1.2 Parts of a DAQ System [[NationalInstruments]] . . . 2
1.3 What Is a Sensor? . . . 2
1.4 What Is a DAQ Device? . . . 2
1.5 What Is a Computer's Role in a DAQ System? . . . 2

2 MERILNE NAPRAVE **3**
2.1 Vernier's DAQs . . . 3
2.2 Arduino Data Acquisition System . . . 5
2.3 Requirements . . . 7
2.4 Frekvenca vzorčenja . . . 8
2.4.1 NALOGA: Frekvenca vzorčenja . . . 9
2.5 Digitalizacija . . . 9
2.6 Resolucija in ločljivost . . . 10
2.6.1 NALOGA: Resolucija AD pretvornika. . . . 10
2.6.2 NALOGA: Ločljivost AD pretvornika. . . . 11
2.7 Točnost in preciznosti (natančnost) . . . 11
2.8 Normalna porazdelitev . . . 12
2.9 Povprečna vrednost . . . 13
2.10 Standardni odklon . . . 13
2.11 Območje zaupanja . . . 13

3 ARDUINO UNO **15**
3.1 Arduino UNO/NANO pinout . . . 16
3.2 Osnove programiranja krmilnika . . . 17
3.2.1 NALOGA: Preskus strojne in programske opreme. . . . 17

4 MERJENJE S KRMILNIKOM ARDUINO UNO **19**
4.0.1 NALOGA: Merjenje napetosti. . . . 19
4.1 Merjenje napetosti . . . 20

4.2 Izračun napetosti . . . 20
4.2.1 NALOGA: Izračun napetosti. . . . 20
4.3 Normalna porazdelitev meritev . . . 20
4.3.1 NALOGA: Normalna porazdelitev meritev in njeni parametri. . . . 20
4.4 Koeficienti normalne porazdelitve . . . 21
4.5 Časovne meritve . . . 22
4.6 Uporaba digitalnih vhodov . . . 22
4.6.1 NALOGA: Časovni odziv človeške reakcije. . . . 23
4.7 Hitrost . . . 23
4.7.1 NALOGA: Merjenje hitrosti s svetlobnimi vrati. . . . 24
4.8 Pospešek . . . 24

5 PRIKAZOVANJE PODATKOV 25
5.0.1 NALOGA: Izpis podatkov. . . . 25
5.1 Serijski izpis . . . 25
5.2 Grafični izris . . . 26
5.3 Izpis na LCD . . . 26
5.4 Izpis na LCD z vzporednim vodilom . . . 28
5.4.1 NALOGA: Izračun vrednosti s pomočjo regresijske premice. . . . 29
5.5 Vezje in zajem podatkov . . . 29
5.6 Linearizacija . . . 30
5.7 Koeficienti regresijske premice . . . 30

6 SENZORJI 33
6.1 Občutljivost . . . 33
6.2 Delilniki napetosti s spremenljivim uporom . . . 33

7 UMERJANJE (TEMPERATURNEGA) SENZORJA 35
7.0.1 NALOGA: Dimenzioniranje senzorja . . . 35
7.1 Umeritveni postopek . . . 36
7.1.1 NALOGA: Umerjanje temperaturnega senzorja. . . . 36
7.2 Interpolacija . . . 38
7.2.1 NALOGA: Interpolacija . . . 38
7.3 Izpis temperature (stand-alone DAQ) . . . 38
7.3.1 NALOGA: Izpis temperature . . . 38

8 PROJEKTNA NALOGA 41
8.0.1 NALOGA: Samostojna postavitev fiz. eksp. s predstavitvijo rezultatov. . . . 41

9 NALOGE **43**
9.0.1 NALOGA: Frekvenca vzorčenja 43
9.0.2 NALOGA: Resolucija AD pretvornika. 43
9.0.3 NALOGA: Ločljivost AD pretvornika. 43
9.0.4 NALOGA: Preskus strojne in programske opreme. 43
9.0.5 NALOGA: Merjenje napetosti. 43
9.0.6 NALOGA: Izračun napetosti. 43
9.0.7 NALOGA: Normalna porazdelitev meritev in njeni parametri. 43
9.0.8 NALOGA: Časovni odziv človeške reakcije. 43
9.0.9 NALOGA: Merjenje hitrosti s svetlobnimi vrati. 43
9.0.10 NALOGA: Izpis podatkov. 43
9.0.11 NALOGA: Izračun vrednosti s pomočjo regresijske premice. 43
9.0.12 NALOGA: Dimenzioniranje senzorja 43
9.0.13 NALOGA: Umerjanje temperaturnega senzorja. 43
9.0.14 NALOGA: Interpolacija 43
9.0.15 NALOGA: Izpis temperature 43
9.0.16 NALOGA: Samostojna postavitev fiz. eksp. s predstavitvijo rezultatov. 43

10 TOČKOVANJE **45**
10.1 Točke:
. 45

VIRI IN LITERATURA **47**

1 MERILNI SISTEMI

Data acquisition is the process of sampling signals that measure real world physical conditions and converting the resulting samples into digital numeric values that can be manipulated by a computer [[Wikipedia]]. Data acquisition systems, abbreviated by the acronyms DAS or DAQ, typically convert analog waveforms into digital values for processing. The components of data acquisition systems include: - Sensors, to convert physical parameters to electrical signals. - Signal conditioning circuitry, to convert sensor signals into a form that can be converted to digital values. - Analog-to-digital converters, to convert conditioned sensor signals to digital values.

Data acquisition applications are usually controlled by software programs developed using various general purpose programming languages such as Assembly, BASIC, C, C++, C#, Fortran, Java, LabVIEW, Lisp, Pascal, etc. Stand-alone data acquisition systems are often called data loggers[[Wikipedia]].

There are also open-source software packages providing all the necessary tools to acquire data from different hardware equipment. These tools come from the scientific community where complex experiment requires fast, flexible and adaptable software. Those packages are usually custom fit but more general DAQ package like the Maximum Integrated Data Acquisition System can be easily tailored and is used in several physics experiments worldwide[[Wikipedia]].

1.1 Importance of real-life experiments for students

- where theory finds its place
- greater motivation for learning
- more (learning) input –> beater understanding

- instant data processing
- explicit presentation of interdependent variables
- visualization of physical quantities that are detected by human nature

1.2 Parts of a DAQ System [[NationalInstruments]]

1.3 What Is a Sensor?

The measurement of a physical phenomenon, such as the temperature of a room, the intensity of a light source, or the force applied to an object, begins with a sensor. A sensor, also called a transducer, converts a physical phenomenon into a measurable electrical signal. Depending on the type of sensor, its electrical output can be a voltage, current, resistance, or another electrical attribute that varies over time. Some sensors may require additional components and circuitry to properly produce a signal that can accurately and safely be read by a DAQ device [[NationalInstruments]]. Kot v enačbi en. **??** in na sliki sl. **??**. (“What Is Data Acquisition? - National Instruments”)

1.4 What Is a DAQ Device?

DAQ hardware acts as the interface between a computer and signals from the outside world. It primarily functions as a device that digitizes incoming analog signals so that a computer can interpret them. The three key components of a DAQ device used for measuring a signal are the signal conditioning circuitry, analog-to-digital converter (ADC), and computer bus. Many DAQ devices include other functions for automating measurement systems and processes. For example, digital-to-analog converters (DACs) output analog signals, digital I/O lines input and output digital signals, and counter/timers count and generate digital pulses[NationalInstruments].

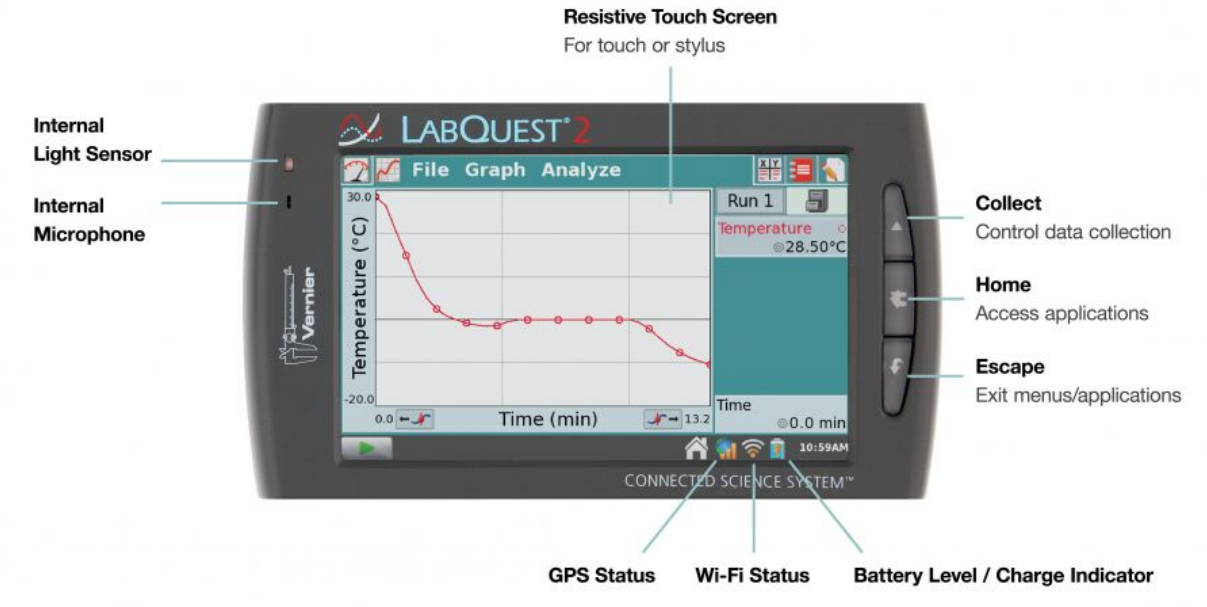
1.5 What Is a Computer’s Role in a DAQ System?

A computer with programmable software controls the operation of the DAQ device and is used for processing, visualizing, and storing measurement data. Different types of computers are used in different types of applications. A desktop may be used in a lab for its processing power, a laptop may be used in the field for its portability, or an industrial computer may be used in a manufacturing plant for its ruggedness (“What Is Data Acquisition? - National Instruments”).

2 MERILNE NAPRAVE

Pri fizikalnih eksperimentih pogosto uporabljamo merilne naprave, s katerimi želimo potrditi teoretične principe in fizikalne pojave. V šolskem prostoru najpogosteje uporabljamo Vernier-jevo merilno napravo LabQuest Hardware “LabQuest® Hardware and Specifications | Vernier” , ki nam omogoča raznovrstne meritve.

2.1 Vernier’s DAQs



Slika 2.1: Vernier-jeva merilna naprava LabQuest2

Poglejmo si nekaj podatkov o napravi:

- **Display**
  - 11.2 cm x 6.7 cm (13.1 cm diagonal) screen

  - 00 x 480 pixel color display at 188 dpi
  - ED backlight
  - ortrait or landscape screen orientation
  - igh-contrast mode for outdoor visibility

- **Processor**
  - 800 MHz Application Processor

- **Connectivity**
  - Wi-Fi 802.11 b/g/n @ 2.4GHz
  - Bluetooth Smart for WDSS and Go Wireless Sensors

- **User Interface**
  - Resistive touch screen
  - Touch and stylus navigation for efficiency and precision

- **Data Acquisition**
  - 100,000 samples per second
  - 12-bit resolution
  - Built-in GPS, 3-axis accelerometer, ambient temperature, light, and microphone

- **Environmental Durability**
  - Operating Temperature: 0 + 45°C
  - Storage Temperature: -30 + 60°C
  - Splash resistant
  - Rugged enclosure designed to withstand a fall from lab bench

- **Size and Weight**
  - Size: 8.8 cm x 15.4 cm x 2.5 cm
  - Weight: 350 g

- **Ports**
  - 5 sensor channels
  - USB port for sensors, flash drives, and peripherals
  - USB mini port
  - DC power jack
  - MicroSD/MMC slot
  - Audio in and out

- **Storage**
  - 200 MB
  - Expandable with MicroSD and USB flash drive
- **Power**
  - Rechargeable, high-capacity battery
  - DC charging/powering through external adapter (included)
- **cena:**
  - $455

Seveda pa morate dokupiti še senzorje, ki tudi niso cenovno ugodni, saj se njihova cena giblje od $30 … naprej.

2.2 Arduino Data Acquisition System

On the market we can find different DAQ systems which are hi-end products and often expencive (from 100 € .. n k€). Buy we can use Arduino (Uno, nano, …) as low-cost data acqusition system if we do not need scientific accurate data (for pedagogical purposes).

<– Parts of DAQ[[NationalInstruments]] –>

Na plošči vsebuje mikrokrmilnik **Atmega328**, ki lahko opravi podobne naloge, kot smo jih opisali v poglavju **Merilne naprave**.

Poglejmo si nekaj karakteristik tega mikrokrmilnika (Atmel 2017):

- **Advanced RISC Architecture**
  - 131 Powerful Instructions
  - Most Single Clock Cycle Execution
  - 32 x 8 General Purpose Working Registers
  - Fully Static Operation
  - Up to 20 MIPS Throughput at 20MHz
  - On-chip 2-cycle Multiplier
- **High Endurance Non-volatile Memory Segments**
  - 32KBytes of In-System Self-Programmable Flash program Memory
  - 1KBytes EEPROM
  - 2KBytes Internal SRAM

  - Write/Erase Cycles: 10,000 Flash/100,000 EEPROM
  - Data Retention: 20 years at 85°C/100 years at 25°C(1)
  - Optional Boot Code Section with Independent Lock Bits
    * In-System Programming by On-chip Boot Program
    * True Read-While-Write Operation
  - Programming Lock for Software Security

- **Atmel® QTouch® Library Support**
  - Capacitive Touch Buttons, Sliders and Wheels
  - QTouch and QMatrix® Acquisition
  - Up to 64 sense channels

- **Peripheral Features**
  - Two 8-bit Timer/Counters with Separate Prescaler and Compare Mode
  - One 16-bit Timer/Counter with Separate Prescaler, Compare Mode, and Capture Mode
  - Real Time Counter with Separate Oscillator
  - Six PWM Channels
  - ADC
    * 8-channel 10-bit ADC in TQFP and QFN/MLF package
    * Temperature Measurement
    * 6-channel 10-bit ADC in PDIP Package
    * 10-bit Resolution
    * 0.5 LSB Integral Non-Linearity
    * ±2 LSB Absolute Accuracy
    * $13 - 260\mu s$ Conversion Time
    * Up to 76.9kSPS (Up to 15kSPS at Maximum Resolution)
    * Six Multiplexed Single Ended Input Channels
    * Two Additional Multiplexed Single Ended Input Channels (TQFP and VFQFN Package only)
    * Temperature Sensor Input Channel
    * Optional Left Adjustment for ADC Result Readout
    * 0 - VCC ADC Input Voltage Range
    * Selectable 1.1V ADC Reference Voltage
    * Free Running or Single Conversion Mode
    * Interrupt on ADC Conversion Complete
    * Sleep Mode Noise Canceler
  - Two Master/Slave SPI Serial Interface
  - One Programmable Serial USART

  - One Byte-oriented 2-wire Serial Interface (Philips I2C compatible)
  - Programmable Watchdog Timer with Separate On-chip Oscillator
  - One On-chip Analog Comparator
  - Interrupt and Wake-up on Pin Change

- **Special Microcontroller Features**
  - Power-on Reset and Programmable Brown-out Detection
  - Internal Calibrated Oscillator
  - External and Internal Interrupt Sources
  - Six Sleep Modes: Idle, ADC Noise Reduction, Power-save, Power-down, Standby, and Extended Standby

- **I/O and Packages**
  - 23 Programmable I/O Lines
  - 28-pin PDIP, 32-lead TQFP, 28-pad QFN/MLF and 32-pad QFN/MLF

- **Operating Voltage:**
  - 1.8 - 5.5V

- **Temperature Range:**
  - -40°C to 105°C

- **Speed Grade:**
  - 0 - 4MHz @ 1.8 - 5.5V
  - 0 - 10MHz @ 2.7 - 5.5V
  - 0 - 20MHz @ 4.5 - 5.5V

- **Power Consumption at 1MHz, 1.8V, 25°C**
  - Active Mode: 0.2mA
  - Power-down Mode: $0.1\mu A$
  - Power-save Mode: $0.0.75\mu A$ (Including 32kHz RTC)

2.3 Requirements

(Checked is required, unchecked is optional)

Hardware:

☒ Computer

- ☒ Arduino board (Arduino UNO, Arduino LEONARDO, Arduino NANO, clone products)
- ☒ basic electronics components for sensors
- ☐ arduino set starter for example use this kit
- ☐ arduino sensors set for example use this kit

Software:

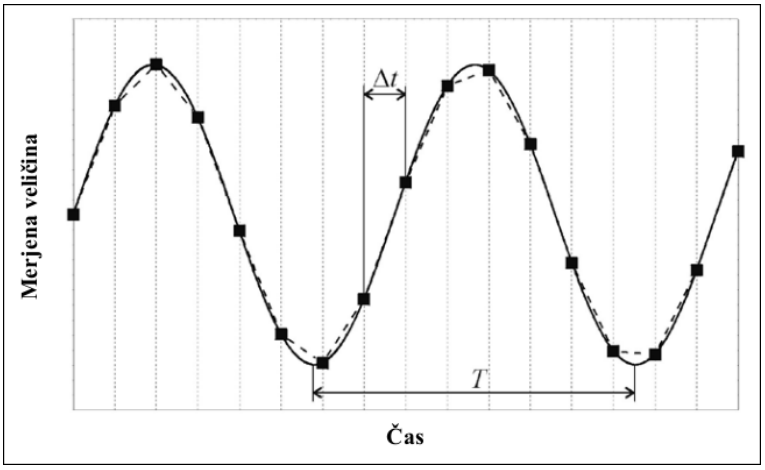
- ☒ Arduino IDE download here
- ☐ Python (>=3.0)
- ☐ pyserial
- ☐ Ms Excel # TEORETIČNE OSNOVE

2.4 Frekvenca vzorčenja

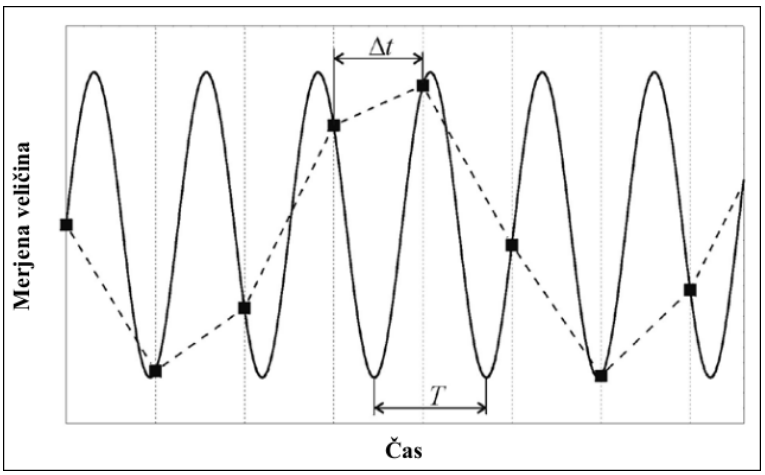
Pri vzorčenju signalov je zelo pomembna frekvenca vzorčenja f_{vz}. Upoštevati moramo Nyqistovo načela vzorčenja, ki pravi, da je potrebno periodične signale vzorčiti vsaj z 2x večjo frekvenco vzorčenja kot je frekvenca signala f_{sig} (Nyquist 1928), kot to predstavlja slika sl. 2.2.

$$f_{vz} = 2f_{sig} \tag{2.1}$$

V nasprotnem primeru laho dobimo nepravilno reprodukcijo merjenega signala (črtkana krivulja), kot to prikazuje slika sl. 2.3.



Slika 2.2: Pravilno vzorčenje sinusnega signala.



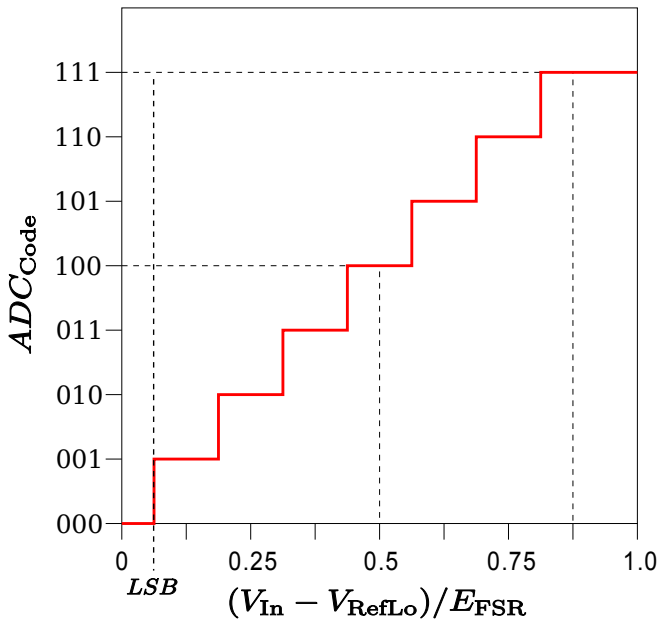
Slika 2.3: Primer reprodukcije (črtkana krivulja) podvzorčenega signala.

2.4.1 NALOGA: Frekvenca vzorčenja

Glede na prejšnje podatke o mikrokrmilniku Atmega328 poiščite podatek o najvišji frekvenci vzorčenja f_{vz} analognih signalov in izračunajte najmanjši čas Δt med dvema vzorčenjema.

2.5 Digitalizacija

Merilni sistemi so opremljeni s t.i. analogno-digitalnimi pretvorniki (ang.: Analog-to-digital converter - ADC), ki pretvarjajo merjeno napetost v neko številsko vrednost. Zelo pogost primer je, ko zvezno napetostno območje od $0,0V - 5,0V$ pretvorimo v številske vrednosti od 0 - 1024. Pri tej pretvorbi ključno vlogo prevzame ADC in njegova **resolucija**. Grafični prikaz take transformacije je prikazan na sliki sl. 2.4.



Slika 2.4: Prenosna funkcija ADC pretvorbe (Contributors 2019a).

2.6 Resolucija in ločljivost

Resolucija AD pretvornikov je določena s številom vseh možnih stanj pretvorbe N. Ker so AD pretvorniki napreave prirejene digitalnim tehnologijam, se njihovi podatki izražajo v dvojiški obliki (binarno). Tako naprimer AD pretvornik z 10-bitno pretvorbo lahko prikaže:

$$N = 2^B \tag{2.2}$$

možnih stanj.

2.6.1 NALOGA: Resolucija AD pretvornika.

Izračunajte s kolikšno resolucijo lahko odčitavamo analogne signale z mikrokrmilnikom Atmega328.

Ločljivost pa je najmanjša razlika med sosednjima digitaliziranima vrednostima merjene količine. Ta vrednost je odvisna tako od števila možnih stanj N, kakor tudi od območja, ki ga pretvarjamo. Zato bi lahko enačbo en. 2.3 zapisali:

$$Locljivost = \frac{Obmocje}{N} \tag{2.3}$$

2.6.2 NALOGA: Ločljivost AD pretvornika.

Izračunajte kolikšna je ločljivost mikrokrmilnika Atmega328 pri odčitavanju analognih signalov.

2.7 Točnost in preciznosti (natančnost)

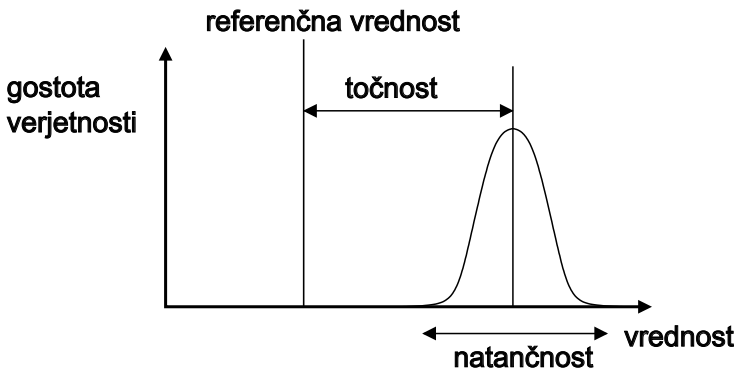
Točnost (v različnih virih je poimenovana različno, ang.: validity) je lastnost merilnega sistema, ki predstavlja ustreznost prestavljene meritve glede na njeno realno merjeno vrednost. Navadno jo izražamo kot relativno napako ϵ v procentualni obliki (enačba en. 2.4):

$$\epsilon = \frac{(X - x_n)}{X} \tag{2.4}$$

Kjer je X realna merjena vrednost in x_n izmerjena vrednost.

Preciznost oz. natančnost (zopet v različnih literaturah poimenovana različno, ang.: reliability) je sposobnost merilnega sistema reprodukcije iste merjene (referncne) vrednosti z enakimi izmerjenimi vrednostmi. V mnogih primerh se izkaže, da gre v tem primeru za naključno napako merjenja in to vrednost lahko ponazarjamo s standardnim odklonom merilnega postopka (enačba en. 2.7).

V splošnem bi lahko točnost in natančnost predstavili z grafom na sliki (Contributors 2019c) sl. 2.5.



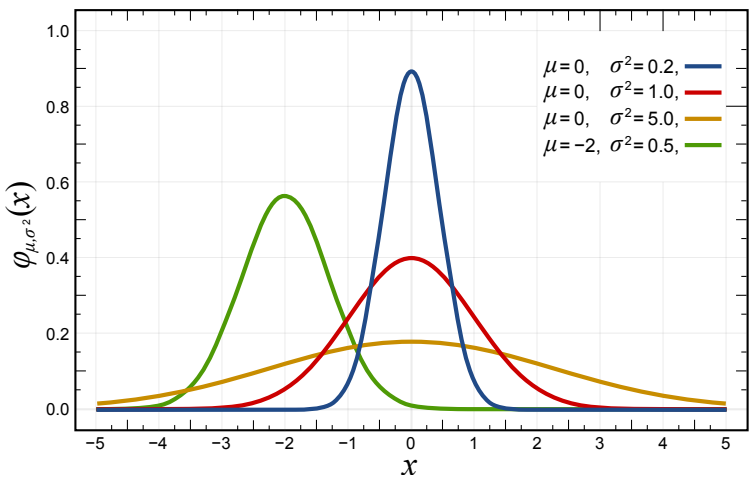
Slika 2.5: Točnost in natančnost mreitev.

2.8 Normalna porazdelitev

Kadar imamo v merilnem sistemu opravka z naključnimi napakami, meritve lahko predstavimo s krivuljo normalne porazdelitve - v splošenm imenovnane Gaussova porazdelitev. Zapišemo jo v obliki enačbe en. 2.5.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{2.5}$$

Kjer je μ povprečna vrednost in σ^2 varianca. Nekaj različnih krivulj lahko vidimo na sliki sl. 2.6 (Contributors 2019b).



Slika 2.6: Primeri normalne verjetnostne porazdelitve.

2.9 Povprečna vrednost

$$\bar{x} = \frac{\sum x_n}{n} \tag{2.6}$$

2.10 Standardni odklon

$$\sigma = \sqrt{\frac{\sum_{n=1}^{N}(x_n - \bar{x})^2}{n-1}} \tag{2.7}$$

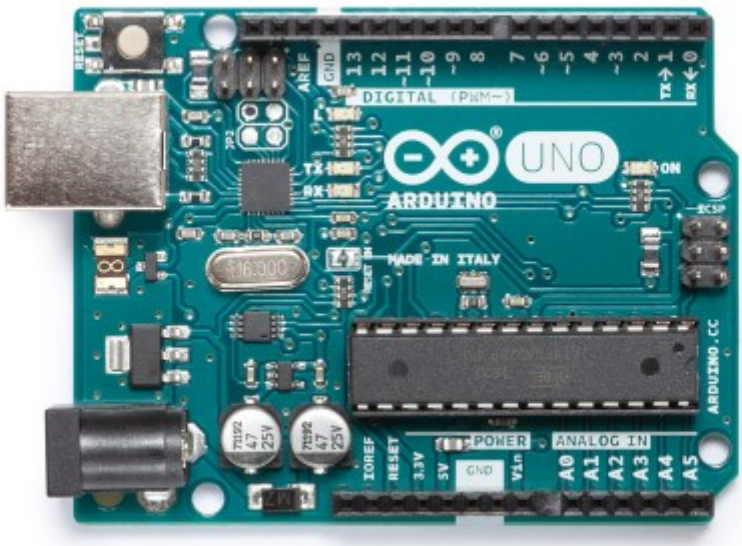
2.11 Območje zaupanja

Z intervalom zaupanja predstavlja območje meritev, v katerem se naključna meritev pojavi z neko verjetnostjo. Pri normali porazdelitvi se izkaže, da je v območju $\bar{x} \pm 1\sigma$ kar 68% vseh meritev, pri $\bar{x} \pm 2\sigma$ jih je 95% in pri $\bar{x} \pm 3\sigma$ celo 99,7%. Tako območje $\pm a\sigma$ imenujemo območje zaupanja. Najpogosteje se v praksi uporablja območje zaupanja s koef. $a = 1,96$, v katerem bomo zanesljivo našli 95,00% meritev.

3 ARDUINO UNO

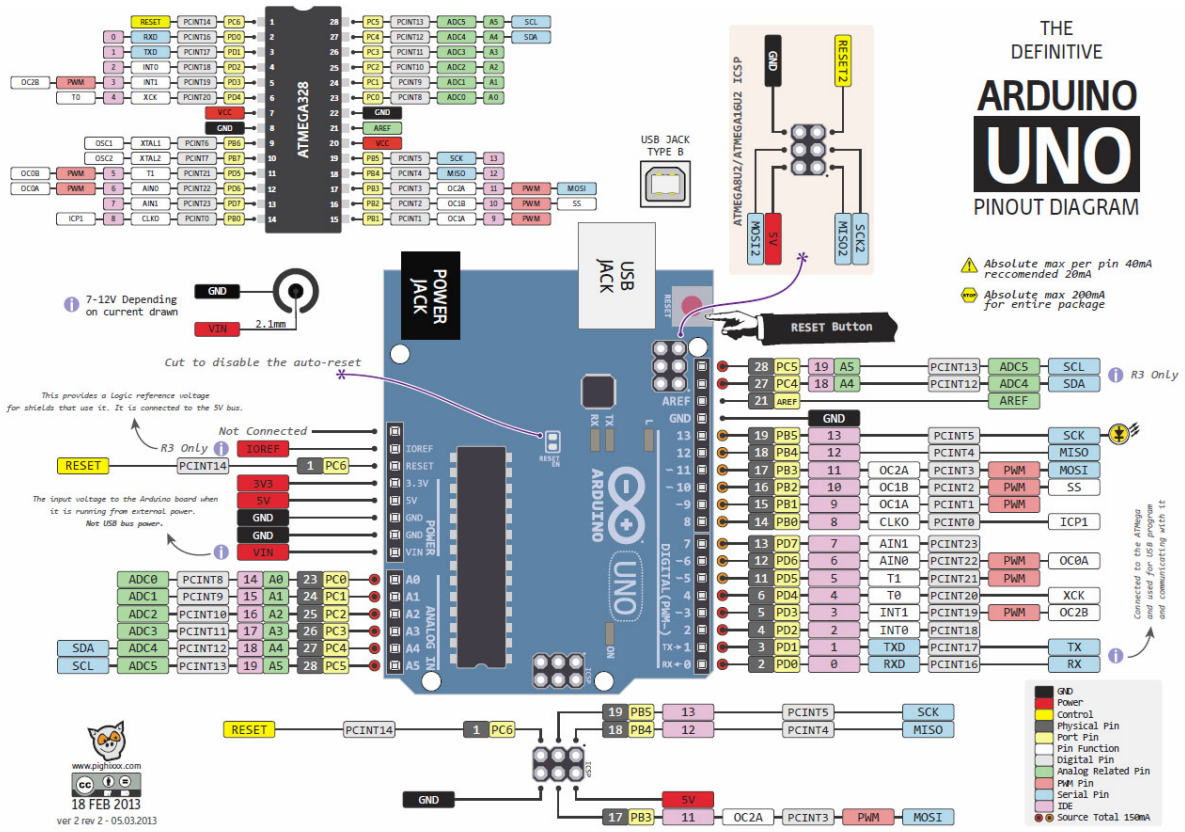
Arduino Uno je:

> The UNO is the best board to get started with electronics and coding. If this is your first experience tinkering with the platform, the UNO is the most robust board you can start playing with. The UNO is the most used and documented board of the whole Arduino family. “Arduino Uno Rev3”

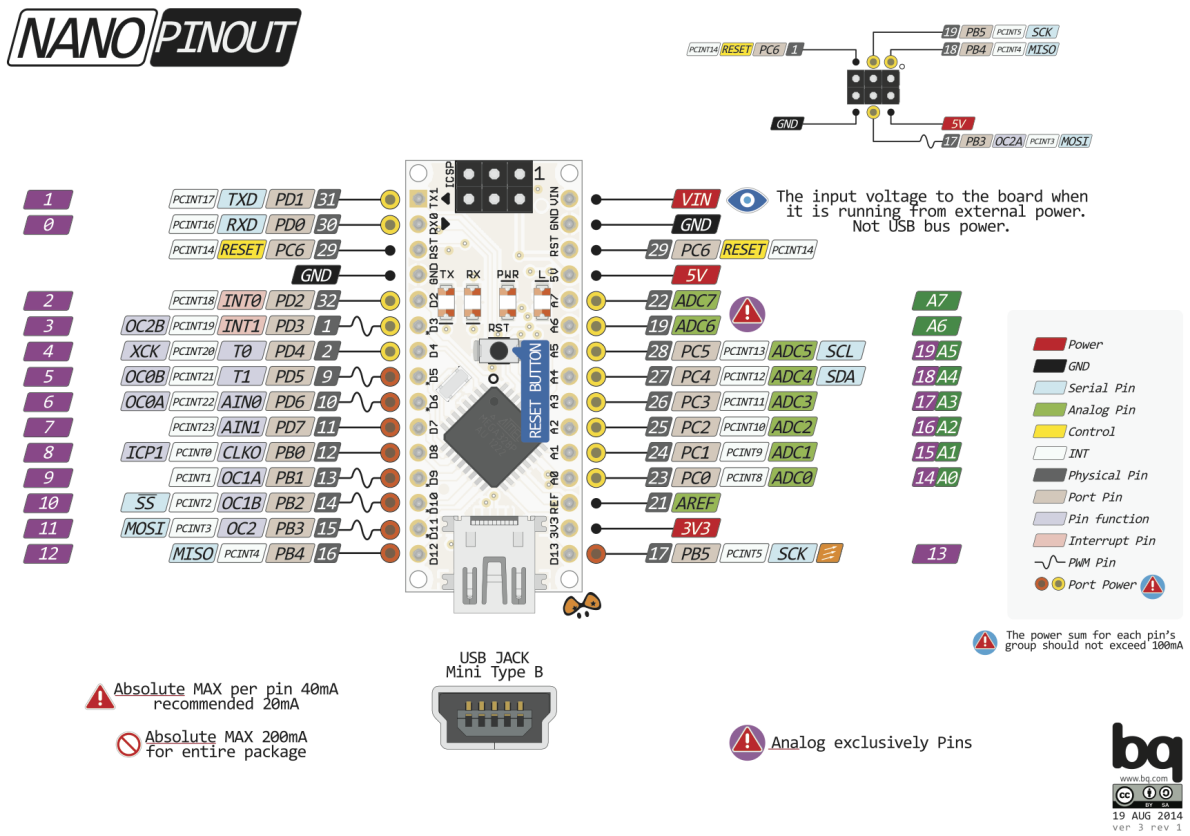


Slika 3.1: Arduino Uno

3.1 Arduino UNO/NANO pinout



Slika 3.2: Razporeditev priključkov na krmilniku Arduino UNO (Theman 2015).



Slika 3.3: Razporeditev priključkov na krmilniku Arduino NANO (Theman 2015).

Kot lahko opazimo, so funkcije priključkov na sliki sl. 3.2 in sliki sl. 3.3 enaki. To ni nenavadno, saj sta krmilnika po električni zgradbi enaka, razlikujeta se le v fizični izvedbi.

3.2 Osnove programiranja krmilnika

3.2.1 NALOGA: Preskus strojne in programske opreme.

Pravilno priključite krminlnik Arduino na računalnik in nastavite programsko okolje Aruino IDE ter sprogramirajte krmilnik s testnim programom **BLINK.INO**. Iz nastavitev programskega okolja prepišite vaše nastavitve za:

1. krmilnik:_____,
2. komunikacijska vrata: _____,
3. mikrokrmilnik: _____.

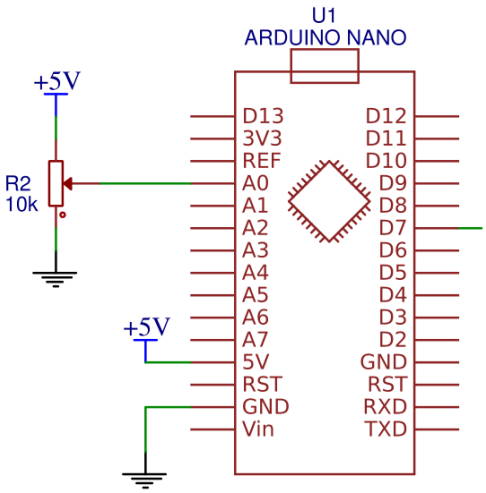
Testni program BLINK.INO

Nastavite in delovanje krmilnika lahko preverimo s testnim programom **blink.ino**. Na krmilniku Arduino UNO je na priključek **13** priključena svetleča dioda prav za ta namen.

```
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

4 MERJENJE S KRMILNIKOM ARDUINO UNO

Krmilnik Atmega328 ima vgrajen ADC pretvornik, s katerim lahko odčitavamo analogne napetosti v območju $U_{ADC} = [0..5]V$.Te napetosti lahko odčitavamo na priključkih krmilnika, ki so označeni z **A0..A7**. Zato moramo senzorje priključiti tako kot prikazuje slika sl. 4.1.



Slika 4.1: priključitev potenciometra na analogni priključek krmilnika.

4.0.1 NALOGA: Merjenje napetosti.

Zvežite vezje po shemi na sliki sl. 4.1 in sprogramirajte krmilnik tako, da boste na ekran računalnika izpisovali izmerjene vrednosti.
Napišite v kakšnem intervalu so bile izmerjene vrednosti:
ADC = [_____,_____]_.

4.1 Merjenje napetosti

```
int adc_value;
void setup() {
    Serial.begin(9600);
}

void loop() {
    adc_value = analogRead(0);
    Serial.println(adc_value);
    delay(100);
}
```

4.2 Izračun napetosti

4.2.1 NALOGA: Izračun napetosti.

Napišite program za merjenje napetosti z ADC vmesnikom. V spodnji prostor pa vpišite le programske vrstice, ki ste jih uporabili za izračun napetosti.

4.3 Normalna porazdelitev meritev

4.3.1 NALOGA: Normalna porazdelitev meritev in njeni parametri.

Z Arduino krmilnikom izmerite 100 meritev neke poljubne napetosti. Nato te meritve preverite še z volt-metrom, kar naj predstavlja vašo referenčno vrednost. Meritve vnesite v program za delo s tabelami in z ustreznimi funkcijami izračunajte, rezultat pa vpišite na črte:

- izmerjena referenčna vrednost: _____,
- povprečno vrednost meritev : _____,
- točnost predstavite z relativno napako :_____,
- preciznost meritev pa podajte s standardnim odklon :_____.

```
void setup() {
  Serial.begin(9600);
  int i = 0;
  int adc_value = 0;
  for (i=0;i<100;i++){
    adc_value = analogRead(0);
    float voltage = (float)adc_value * 5 / 1024;
    Serial.println(voltage, 4);
    delay(10);
  }
}
void loop() {
}
```

4.4 Koeficienti normalne porazdelitve

Sploščenost

```
=KURT(Range)
```

Premaknjenost

```
=SKEW(Range)
```

Povprečna vrednost

```
=AVERAGE(Range)
```

Standardna napaka

```
=STEYX(y-Range,x-Range)
```

Interval zaupanja

```
=CONFIDENCE(Signif., Std.Dev., Sample Size)
```

4.5 Časovne meritve

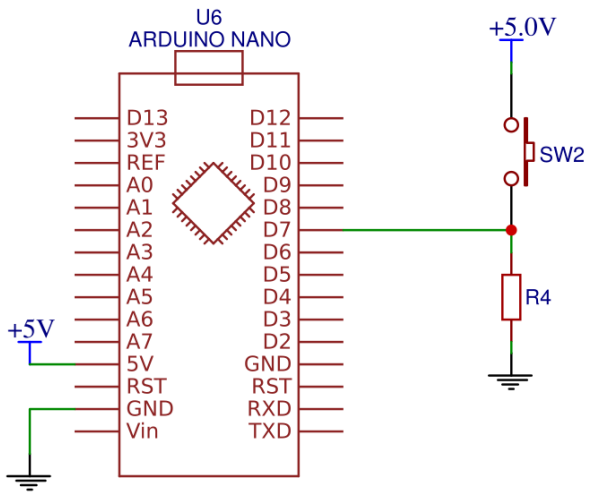
```
unsigned long time;

void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.print("Time: ");
  time = millis();

  Serial.println(time); //prints time since program started
  delay(1000);          // wait a second so as not to send massive
      amounts of data
}
```

Preverite tudi:

```
time = micros();
```

4.6 Uporaba digitalnih vhodov

V povezavi s časovnimi meritvami pogosto uporabljamo digitalne vhode. Le-te lahko najdemo na priključkih **D0..D13** in tudi na **A0..A7**. Primer enostavne vezave tipke na krmilnik prikazuje slika sl. 4.2.



Slika 4.2: Vezava tipke na digitalni vhod krmilnika Arduino.

4.6.1 NALOGA: Časovni odziv človeške reakcije.

Napišite program za merjenje hitrosti človeškega odziva in zvežite vezje, ki ga prikazuje slika sl. 4.2. Po naključnem času naj zasveti lučka na krmilniku ArduinoUNO. Nato pa naj program nemudoma shrani trenutni čas v $start_time$ in ko uporabnik pritisne tipko naj si program ponovno shrani čas v $stop_time$. Program naj izračuna razliko časov in ga prikaže v na računalniku.
Napredno: Če znate program popravite tako, da ne bo omogočal goljufanja.

```
void setup() {
  pinMode(13,OUTPUT);
  pinMode(7,INPUT);
  Serial.begin(9600);
  Serial.println("Start...");
  randomSeed(analogRead(0));
}

void loop() {
  digitalWrite(13,LOW);
  delay(random(5000,10000));
  digitalWrite(13,HIGH);

  unsigned long start_time = micros();
  unsigned long stop_time = 0;

  while (digitalRead(7)==0){
    stop_time = micros();
  }

  unsigned long time_div = stop_time - start_time;
  Serial.println(time_div);

  while (digitalRead(7)==1){
    delay(200);
    Serial.println("spusti tipko...");
  }
}
```

4.7 Hitrost

4.7.1 NALOGA: Merjenje hitrosti s svetlobnimi vrati.

Uporabite ali sestavite dvojna svetlobna vrata. Nato napišite program, ki bo izmeril razliko v času ko se svetlobni snop na enih in drugih svetlobnih vratih prekine. Izmerite razdaljo med vratoma in izračunajte hitrost.
Napredno: Če zante program popravite tako, da bo občutljiv na “spremembo” vhodnega signala.

```
void setup() {
  pinMode(7,INPUT);
  pinMode(8,INPUT);
  Serial.begin(9600);
  Serial.println("Start...");
}

void loop() {
  unsigned long start_time = 0;
  unsigned long stop_time = 0;

  while (digitalRead(7)==0){
    start_time = micros();
  }

  while (digitalRead(8)==0){
    stop_time = micros();
  }

  unsigned long time_div = stop_time - start_time;
  Serial.println(time_div);

  Serial.println("Nova maritev...");
}
```

4.8 Pospešek

5 PRIKAZOVANJE PODATKOV

Prikazovanje podatkov je ena od ključnih nalog vseh merilnih sistemov. To pahko naredimo na različne načine:

1. v obliki tabele na zaslon računalnika,
2. v grafični obliki,
3. na zaslon samostojne merilne naprave ...

5.0.1 NALOGA: Izpis podatkov.

Prikažite podatke na različne načine:
1. v obliki tabele,
2. v grafični obliki in
3. na LCD zaslon merilne naprave.

5.1 Serijski izpis

Spodnji primer programske kode prikazuje kako lahko z Arduino krmilniki izpisujemo osnovne podatke neke meritve. Izpisovali bomo dva podatka v ločena stolpca. Stolpca bosta ločena s tabulatorjem, kar predstavlja označba **\t**. V prvem stolpcu bomo izpisovali trenutni čas (**millis();**), v drugega pa vrednost analogne pretvorbe (**analogRead(0);**).

```
void setup() {
  Serial.begin(9600);
  Serial.println("t[ms]\tADC");
}

void loop() {
  Serial.print(millis());
  Serial.print("\t");
  Serial.println(analogRead(0));
  delay(100);
}
```

5.2 Grafični izris

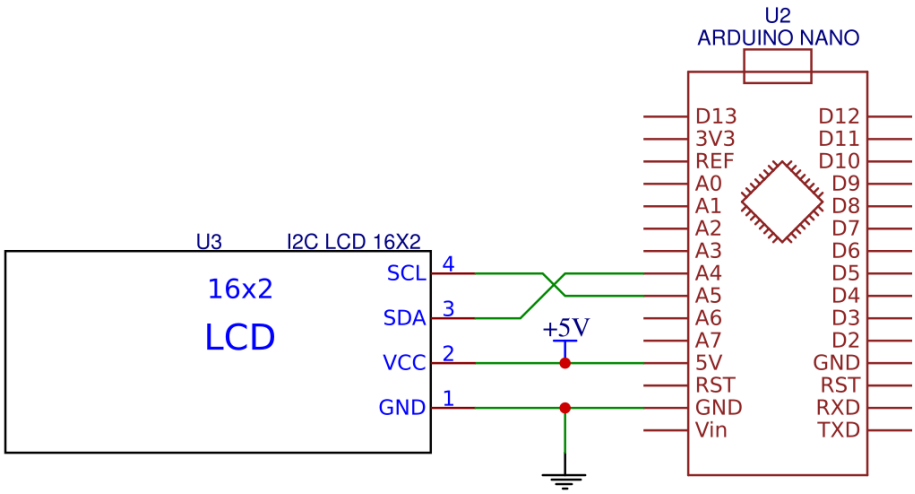
Zelo nazorno je spremljati izris podatkov v grafičnem načinu. Do neke mere nam to funkcijo ponuja programsko okolje Arduino IDE, vendar v zelo okenjeni obliki. Preskusite naslednji program:

```
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.print("0,1024,");
  Serial.println(analogRead(A0));
  delay(10);
}
```

Kot lahko opazimo računalniku pošiljamo 3 različne števila: 0, 1024 in ADC vrednost. Prvi dve nam slušita kot območje, saj bi v nasprotnem primeru program sam prilagajal skalo na y osi (ang.: autofit).

5.3 Izpis na LCD

Če želimo narediti merilno napravo, ki bo delovala samostojno, bomo verjetno potrebovali dodaten ekran za prikaz izmerjenih vrednosti. Zato ga moramo priključiti na krmilnik Arduino, kot kaže slika sl. 5.1.



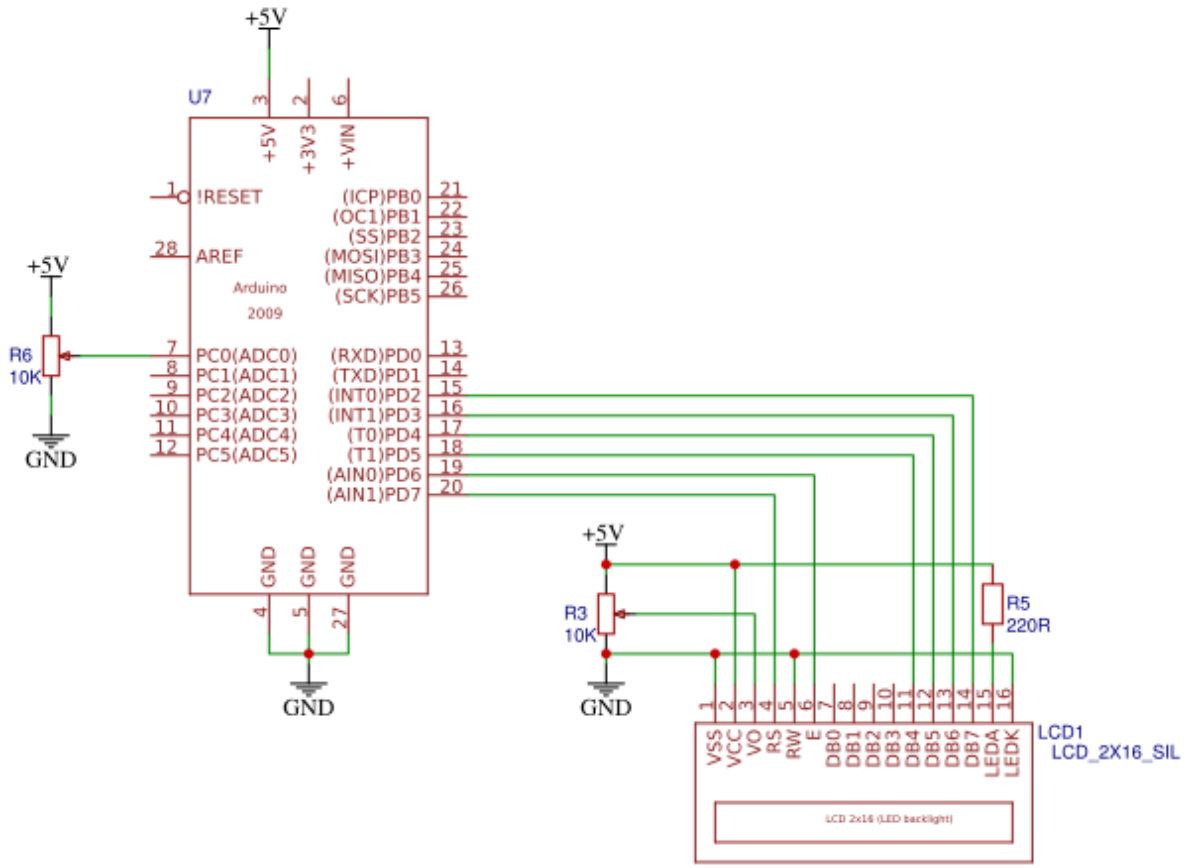
Slika 5.1: Vezava I²C LCD-ja na krmilnik Arduino.

Ena naj enostavnejših rešitev je, da krmilniku Arduino UNO dodamo LCD ekranček z I²C vodilom. Ker je I²C vodilo zasnovano tako, da nanj lahko priključimo več naprav, moramo za vsako napravo

poznati njen naslov. Le tega lahko pridobimo od proizvajalca naprave, ga nastavimo sami ali pa z ustreznim programom pregledamo vse naslove priključenih naprav (programska koda(Arduino 2019)). Za uporabo LCDja na podatkovnem vodilu I^2C bomo potrebovali knjižnico **LiquidCrystal_I2C**, ki si jo lahko presnamete s spletne strani Francisco Malpartida.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C DaqLcd(0x27, 2, 1, 0, 4, 5, 6, 7, 3, POSITIVE);

void setup() {
  DaqLcd.begin(8, 2);
}

void loop() {
  int i = 0;
  long ch = 0;
  for (i=0;i<64;i++){
    ch += analogRead(7);
  }

  float val = ch/64.0;
  DaqLcd.clear();
  DaqLcd.print(val);
}
```

5.4 Izpis na LCD z vzporednim vodilom



Slika 5.2: Shema vezave LCDja z vzporednim vodilom.

Vezavo žic na LCD si lahko prilagodite in vrednosti priključkov popravite v programski kodi.

```
#include <LiquidCrystal.h>
//            lcd(RS, EN, D4,D5,D6,D7);
LiquidCrystal lcd(8,  9,  10,11,12,13);

void setup() {
  lcd.begin(16, 2);
  lcd.print("Voltage:");
}

void loop() {
  lcd.setCursor(0, 1);
  lcd.print(analogRead(A0));
}
```

V program smo vključili knjižnico LiquidCrystal.h, iz katere lahko uporabljamo funkcije kot so:
- LiquidCrystal() - begin() - clear() - home() - setCursor() - write() - print() - cursor() - noCursor() - blink() - noBlink() - display() - noDisplay() - scrollDisplayLeft() - scrollDisplayRight() - autoscroll() - noAutoscroll() - leftToRight() - rightToLeft() - createChar() # LINEARIZACIJA MERITVENIH VREDNOSTI (PRAZNENJE KONDENZATORJA)

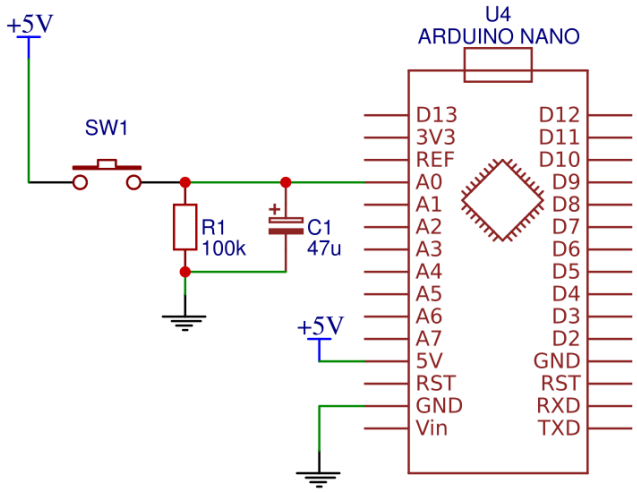
Izvedli bomo poskus, pri katerem bomo lahko meritve odvisne spremenljivke (napetost) opravili pri različnih vrednostih neodvisne spremenljivke (čas). Iz rezultatov bomo skušali izluščiti kapaciteto kondenzatorja (kot iskano konstatnto sistema). Ker je fizikalni pojav nelinearen, ga bomo najprej linearizirali, in šele nato izračunali iskano konstanto (kapaciteto kondenzatorja). Tako lahko v izračun enostavno vključimo vse meritve…

5.4.1 NALOGA: Izračun vrednosti s pomočjo regresijske premice.

Izvedite poizkus praznjenja kondenzatorja in izmerite $U_C(t)$. Podatke linearizirajte tako, da jim boste lahko dodelili regresijsko premico.
Izračunajte:
- resresijski koeficient: $R\check{s}$:_____,
- enačba smernega koeficienta lienaricacije:
- vrednost smernega koeficienta:
- vrednost kapacitete kondenzatorja:

5.5 Vezje in zajem podatkov

Vezje sestavimo po sliki sl. 5.3:



Slika 5.3: Vezava vezja za zajem napetosti na kondenzatorju.

5.6 Linearizacija

$$f(x) = kx + n \tag{5.1}$$

$$U_C(t) = U_0 e^{-\frac{t}{RC}} \tag{5.2}$$

$$lnU_0 = -\frac{1}{RC}t + lnU_C \tag{5.3}$$

ali ...

$$ln(\frac{U_C}{U_0}) = -\frac{1}{RC}t \tag{5.4}$$

5.7 Koeficienti regresijske premice

```
=SLOPE(y-Range,x-Range)
```

```
=INTERCEPT(y-Range,x-Range)
```

```
Korelacija med x in y.
=CORREL(y-Range,x-Range)
```

```
Korelacija med y_0 in y.
=RSQ(y-Range,x-Range)
```

6 SENZORJI

- toleranca uporov

6.1 Občutljivost

Ob predpostavki, da je določena meritev linearna na celotnem merilnem področju merilnega instrumenta, lahko izrazimo občutljivost instrumenta kot kvocient med spremembo izmerjene in spremembo merjene spremenljivke. Občutljivost analognega instrumenta določa razmerje med linearnim pomikom indikatorja lege in spremembo merjene spremenljivke, ki omenjeni pomik povzroči.

6.2 Delilniki napetosti s spremenljivim uporom

Pri takih vrstah senzorjev je izrednega pomena, da izberemo primeren refernčni upor R_{Ref}, saj le-ta vpliva na občutljivost senzorja. Izhodno napetost za temperaturni senzro s termistorjem in referenčnim uporom lahko izračunamo po enačbi en. 6.1.

$$U_{izh} = \frac{U_0 R_{Ref}}{R_{Ref} + R_{NTC}} \tag{6.1}$$

Če bomo ta temperaturni senzor uporabljali v temperaturnem intervalu $T \in [T_{min}, T_{max}]$, se bo njegova upornost spreminjala v intervalu $R_{NTC} \in [R_{NTC_{max}}, R_{NTC_{min}}]$. Ker želimo, da bi senzor imel v tem območju kar največjo občutljivost lahko sestavimo enačbo en. 6.2, za spremembo izhodne napetosti pri intervalu T.

$$\Delta U_{izh} = U_{izh}(T_{max}) - U_{izh}(T_{min}) \tag{6.2}$$

Če poiščemo maksimum funkcije enačbe en. 6.2 za R_{Ref} se izkaže, da je največja občutljivost senzorja pri refernčnem uporu, ki ga podaja enačba en. 6.3:

$$R_{Ref} = \sqrt{R_{NTC_{min}} R_{NTC_{max}}} \tag{6.3}$$

Delilnik napetosti z notranjim uporom

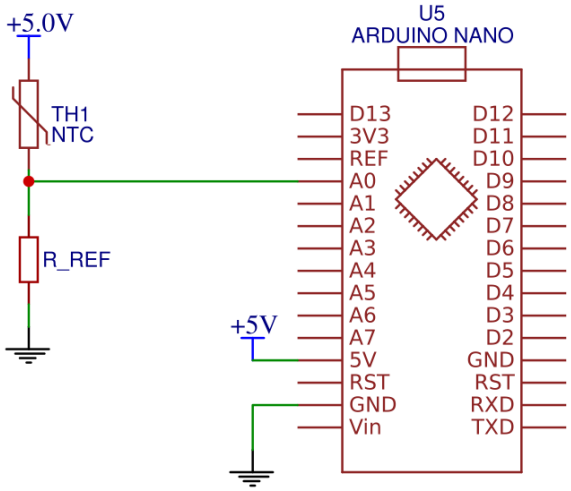
Delilnik napetosti lahko Naredimo tudi z uporom, ki ga mikrokrmilnik že vsebuje. Ker je ta upor vezan proti napajanju ga imenujemo **pull-up resistor**. V našem primeru ima ta upor upornost $R_{PULL-UP} \approx 10k\Omega$. Ker njegova vrednost ni točna je manj primeren za uporabo pri analognih senzorjih, vendar za izdelavo digitalnih lahko nadomesti nepotrebne vezave.

7 UMERJANJE (TEMPERATURNEGA) SENZORJA

- glej poglavje o delilnikih
- naredite temp. senzor, katerega odziv bo največji za temperaturno območje $T_{NTC} = [15, 95]$řC.

7.0.1 NALOGA: Dimenzioniranje senzorja

Podrobno si poglejte sec. **??** in dimenzionirajte temperaturn sentor z NTC termistorjem za območje $T = [15, 95]$řC. Izračunajte ali izmerite ključne vrednosti za dimenzioniranje senzorja:
$R_{NTC,T=15řC} =$_____,
$R_{NTC,T=95řC} =$_____,
$R_{REF} =$_____.



Slika 7.1: Vezava temperatunega senzorja.

7.1 Umeritveni postopek

7.1.1 NALOGA: Umerjanje temperaturnega senzorja.

Naredite umeritveni postopek in predstavite tabelo s podatki.

T[°C] | **U[V]**

```
String inputString = "";         // a String to hold incoming data
boolean stringComplete = false;  // whether the string is complete

void setup() {
  Serial.begin(9600);
  printHelp();
  inputString.reserve(20);
}
void printHelp(){
  Serial.println("##########################################");
  Serial.println("# INSTRUCTIONS:                          #");
  Serial.println("#   1. Insert temperature (example: 22.4) #");
  Serial.println("#   2. then press ENTER                  #");
  Serial.println("#   3. repeat steps 1. and 2. ...        #");
  Serial.println("# When you are done with calibration:    #");
  Serial.println("#   4. Copy data to spreadsheet.         #");
  Serial.println("#   5. Add some polynome to the points.  #");
  Serial.println("#   6. Insert coeficients into file:     #");
  Serial.println("#       'temperature.ino'                #");
  Serial.println("##########################################");
  Serial.println("T°[C] Value");
}
void loop() {
  //float ref_temp = waitForUserInput();
  //float sen_val = sensorsAvgValueOnCh(A0);
  //printNewTabelRow(ref_temp, sen_val);
  if (stringComplete) {
    Serial.print(inputString);
    Serial.print(" ");
    inputString = "";
    stringComplete = false;

    long CH1 = 0;

    int i;
    for (i = 0; i < 128; i++) {
      CH1 += analogRead(A7);
      delay(1);
    }
    float Vol_CH1 = float(CH1/128.0);          // calibration in ADC values...
    //float Vol_CH1 = CH1*5.0/1024; // calibration in Voltage...
    Serial.println(Vol_CH1);
  }
}
float waitForUserInput(){
  char inChar = '';
  while (inChar != '\n'){
    if (Serial.available()) inChar = (char)Serial.read();
    if (inChar != '\n')
  }
}
void serialEvent() {
  while (Serial.available()) {
    char inChar = (char)Serial.read();  // get the new byte:
    if (inChar == '\n') {
      stringComplete = true;
    } else {
      inputString += inChar;              // add it to the inputString:
    }
  }
```

7.2 Interpolacija

7.2.1 NALOGA: Interpolacija

Dobljene podatke vnesite v programsko orodje za delo s tabelami in podatke izrišite v grafični obliki. Nato dodajte ustrezen trend in izpišite enačbo prenosne funkcije.
Prenosna funkcija:

7.3 Izpis temperature (stand-alone DAQ)

7.3.1 NALOGA: Izpis temperature

V programskem orodju za tabele (Microsoft Excel ali LibreOffice Calc) podatkom priredimo ustrezno krivuljo in odčitamo koeficiente… Nato napišemo program:

```
//            k0        k1        k2        k3        k4      k5
float k[6] = { -74.91, 530.963E-3, -1.685E-3, 3.253E-6, -3.128E-9, 1.227
    E-12};

void setup() {
  pinMode(15, OUTPUT);
  digitalWrite(15, HIGH);
  pinMode(16, OUTPUT);
  digitalWrite(16, LOW);
  pinMode(14, INPUT); //A0
  // initialize the serial communication:
  Serial.begin(9600);
}

void loop() {
  // send the value of analog input 0:
  int i;
  float CH1 = 0;
  for (i = 0; i < 64; i++) {
    CH1 += analogRead(A7);
    delay(3);
  }
  float Vol_CH1 = float(CH1 / 64.0);

  float Temp_CH1 = 0;
  for (int i = 0; i <= 5; i++) {
    Temp_CH1 += k[i] * pow(Vol_CH1, i);
  }
  Serial.println(Temp_CH1);
  //delay(1000);
}
```

8 PROJEKTNA NALOGA

8.0.1 NALOGA: Samostojna postavitev fiz. eksp. s predstavitvijo rezultatov.

Sestavite poljuben fizikalni eksperiment, ga opremite s primernimi senzorji in podatke zajemite s krmilnikom Arduino. V primernim programskem okolju podatke obdelajte in jih številsko in grafično prikažite.
Dokumentirajte:
1. kratek opis eksperimenta,
2. vezje,
3. postavitev eksperimenta,
4. programsko kodo,
5. meritve,
6. obdelavo podatkov in
7. predstavitev rezultatov.

9 NALOGE

9.0.1 NALOGA: Frekvenca vzorčenja

9.0.2 NALOGA: Resolucija AD pretvornika.

9.0.3 NALOGA: Ločljivost AD pretvornika.

9.0.4 NALOGA: Preskus strojne in programske opreme.

9.0.5 NALOGA: Merjenje napetosti.

9.0.6 NALOGA: Izračun napetosti.

9.0.7 NALOGA: Normalna porazdelitev meritev in njeni parametri.

9.0.8 NALOGA: Časovni odziv človeške reakcije.

9.0.9 NALOGA: Merjenje hitrosti s svetlobnimi vrati.

9.0.10 NALOGA: Izpis podatkov.

9.0.11 NALOGA: Izračun vrednosti s pomočjo regresijske premice.

9.0.12 NALOGA: Dimenzioniranje senzorja

9.0.13 NALOGA: Umerjanje temperaturnega senzorja.

9.0.14 NALOGA: Interpolacija

9.0.15 NALOGA: Izpis temperature

9.0.16 NALOGA: Samostojna postavitev fiz. eksp. s predstavitvijo rezultatov.

10 TOČKOVANJE

Vse naloge od 1 .. 15 se točkujejo s $t_{1..15} = [0, 1, 2]$ točke.
Naloga 16 pa se točkuje s $t_{16} = [0, 5, 10]$ točkami za vsako od postavk:

Naloga: **16.1** (kratek opis eksperimenta) =
Naloga: **16.2** (vezje) =
Naloga: **16.3** (postavitev eksperimenta) =
Naloga: **16.4** (programsko kodo) =
Naloga: **16.5** (meritve) =
Naloga: **16.6** (obdelavo podatkov in) =
Naloga: **16.7** (predstavitev rezultatov =

10.1 Točke:

$t_{1..15} =$..... $/30$
$t_{16} =$..... $/70$
Skupaj:
$t_{RPF} =$..... $/100$

VIRI IN LITERATURA

Arduino. 2019. “Arduino Playground - I2cScanner.” 2019. https://playground.arduino.cc/Main/I2cScanner/.

“Arduino Uno Rev3.” https://store.arduino.cc/usa/arduino-uno-rev3.

Atmel. 2017. “Atmega328p.” 2017. http://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

Contributors, Wikipedia. 2019a. “Analog-to-Digital Converter.” 2019. https://en.wikipedia.org/wiki/Analog-to-digital_converter.

———. 2019b. “Normal Distribution.” 2019. https://en.wikipedia.org/wiki/Normal_distribution.

———. 2019c. “Točnost in Natančnost.” 2019. https://sl.wikipedia.org/wiki/To%C4%8Dnost_in_natan%C4%8Dnost.

“LabQuest® Hardware and Specifications | Vernier.” https://www.vernier.com/products/interfaces/labq/hardware/.

Nyquist, H. 1928. “Certain Topics in Telegraph Transmission Theory.” *Transactions of the American Institute of Electrical Engineers* 47 (2): 617–44. https://doi.org/10.1109/t-aiee.1928.5055024.

Theman, Dan. 2015. “Can’t Get I2c to Work on an Arduino Nano? (Pinout Diagrams).” 2015. https://bigdanzblog.wordpress.com/2015/01/30/cant-get-i2c-to-work-on-an-arduino-nano-pinout-diagrams.

“What Is Data Acquisition? - National Instruments.” http://www.ni.com/data-acquisition/what-is/.